

$$\begin{aligned}
\mathcal{I}(\rho) &= I(s; \sqrt{\rho}s + z) \\
&= \mathfrak{h}(\sqrt{\rho}s + z) - \mathfrak{h}(\sqrt{\rho}s + z|s) \\
&= \mathfrak{h}(\sqrt{\rho}s + z) - \mathfrak{h}(z) \\
&= \log_2(\pi e(1 + \rho)) - \log_2(\pi e) \\
&= \log_2(1 + \rho).
\end{aligned} \tag{1.51}$$

Relevant comments:

- In $\mathfrak{h}(\sqrt{\rho}s + z|s) = \mathfrak{h}(z)$, we know that we've already fixed s : s multiplied by whichever scalar won't bring any extra information than z , as we already know what s is; for this reason, the differential entropies are the same.

If we then consider $x \rightarrow 0$, we have

$$\log_e(1+x) \xrightarrow{x \rightarrow 0} x - \frac{1}{2}x^2 + o(x^2)$$

So we need to **convert to base e** and we'll then be able to use the expansion:

$$= \left(e - \frac{1}{2}e^2\right) \log_2 e + o(e^2)$$

And **for large ρ** we'll then have

$$\xrightarrow{\rho \rightarrow +\infty} \log_2(\rho) + O(1/\rho)$$

Note

This is **crucially important** as it allows to model an AWGN channel! It allows us to define the capacity as

$$\frac{1}{2} \log(1 + \text{SNR})$$

(we don't have $1/2$ since this is complex; we'd have $1/2$ if it wasn't complex!).

MIMO intuition: we'll use random vectors in order to model MIMO communications

Our book is MIMO! For a long time we had a setup with a transmitter and a receiver.

People asked: what can we do in order to increase the spectral efficiency of our signals? The spectral efficiency is how many bit/s/Hz we can squeeze.

The spectral efficiency we've reached is between 1 and 5 bit/s/Hz; we generally have 4 bit/s/Hz.

$B \cdot \nu$ is how many bits we can send (B = bandwidth, ν = spectral efficiency).

What if we have two antennas at the transmitter and two at the receiver? It's been shown that by doing such, **it's like having twice the bandwidth**: this is the whole concept of MIMO.

Of course we have to pay *double*.

With two antennas we're transmitting two symbols, aka a vector:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = A \begin{bmatrix} s_1 \\ s_2 \end{bmatrix} + \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

We thus need to use vectors, we can't stay with scalars!

Mutual information of a gaussian vector through an AWGN channel (+figure of noise)

Let's say we have a vector s and a matrix A (which is needed since every signal is communicating with each antenna!): **we're now in the MIMO setting**.

We have a Gaussian vector s , a matrix A and a noise Gaussian vector z ; we'll have an output

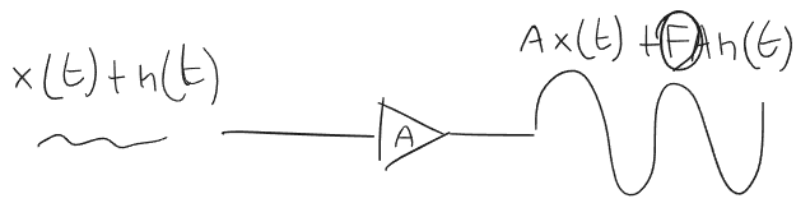
$$y = \sqrt{\rho} A s + z,$$

where A is considered deterministic (we'll see it's not really true), z is the noise, which originates from **thermal noise**: electrons are not *still*; the more we increase the temperature, the more electrons move; we want to transmit a certain voltage, but we'll measure a certain fluctuating value added to our transmitted signal: the fluctuating addition comes from thermal noise!

✍ Noise figure

What about **noise figure**?

When we amplify a signal, do we pay a price? We're amplifying the noise as well, but we might amplify the noise more: the amplifier will add its own noise to the noise, which is F , the noise figure:



We now want to calculate the mutual information between source and output; the idea is the same, only with matrices and vectors; we have

$$\mathcal{I}(\rho) = I(\mathbf{s}; \sqrt{\rho}\mathbf{A}\mathbf{s} + \mathbf{z}) \quad (1.74)$$

$$= \mathfrak{h}(\sqrt{\rho}\mathbf{A}\mathbf{s} + \mathbf{z}) - \mathfrak{h}(\sqrt{\rho}\mathbf{A}\mathbf{s} + \mathbf{z}|\mathbf{s}) \quad (1.75)$$

$$= \mathfrak{h}(\sqrt{\rho}\mathbf{A}\mathbf{s} + \mathbf{z}) - \mathfrak{h}(\mathbf{z}) \quad (1.76)$$

$$= \log_2 \det(\pi e(\rho\mathbf{A}\mathbf{R}_s\mathbf{A}^* + \mathbf{R}_z)) - \log_2 \det(\pi e\mathbf{R}_z) \quad (1.77)$$

$$= \log_2 \det(\mathbf{I} + \rho\mathbf{A}\mathbf{R}_s\mathbf{A}^*\mathbf{R}_z^{-1}), \quad (1.78)$$

with (1.77) following from the definition of **differential entropy of a gaussian vector**.

The correlation matrix of $\sqrt{\rho}\mathbf{A}\mathbf{s}$ is $\rho\mathbf{A}\mathbf{R}_s\mathbf{A}^*$; we then have a sum, so we'll have $\mathbf{R}_z$.

The fact that in the last passage (1.78) we have $\mathbf{R}_z$ at the denominator, means we'll be multiplying by $\mathbf{R}_z^{-1}$; this follows from the fact that

$$\frac{\det(\mathbf{A})}{\det(\mathbf{B})} = \det(\mathbf{A}\mathbf{B}^{-1}),$$

here applied with $\mathbf{A} = \rho\mathbf{A}\mathbf{R}_s\mathbf{A}^* + \mathbf{R}_z$ and $\mathbf{B} = \mathbf{R}_z$.

How do we know that $\sqrt{\rho}\mathbf{A}\mathbf{s} + \mathbf{z}$ is Gaussian?

I think the idea is that we can consider $\sqrt{\rho}\mathbf{A}\mathbf{s} + \mathbf{z}$ as a multiplication between the matrix

$$[\sqrt{\rho}\mathbf{A} \quad \mathbf{1}]$$

and the vector

$$\begin{bmatrix} \mathbf{s} \\ \mathbf{z} \end{bmatrix},$$

which will result in a random vector with distribution

$$\sim \mathcal{N}_{\mathbb{C}} \left([\sqrt{\rho}\mathbf{A} \quad \mathbf{1}] \cdot \begin{bmatrix} 0 \\ 0 \end{bmatrix}, [\sqrt{\rho}\mathbf{A} \quad \mathbf{1}] \cdot \begin{bmatrix} \mathbf{R}_s & 0 \\ 0 & \mathbf{R}_z \end{bmatrix} \cdot \begin{bmatrix} \sqrt{\rho}\mathbf{A}^* \\ 1 \end{bmatrix} \right),$$

according to

Transformation of a Gaussian vector

If we have:

- A Gaussian vector $\mathbf{x} \sim \mathcal{N}_{\mathbb{C}}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ of dimensions $N \times 1$,
- a matrix $\mathbf{A}$ of dimensions $M \times N$
- a vector $\mathbf{b}$ of dimensions $M \times 1$

we have that the vector

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$$

will be distributed according to

$$\mathbf{y} \sim \mathcal{N}_{\mathbb{C}}(\mathbf{A}\boldsymbol{\mu} + \mathbf{b}, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^*) .$$

🦋 Extension of **Uni generale**.

📄 Derivative of logarithm of determinant

In order to go on, we consider the following relations:

$$\begin{aligned} \frac{\partial}{\partial \rho} \log_e \det(\mathbf{I} + \rho \mathbf{B}) \Big|_{\rho=0} &= \text{tr}(\mathbf{B}) \\ \frac{\partial^2}{\partial \rho^2} \log_e \det(\mathbf{I} + \rho \mathbf{B}) \Big|_{\rho=0} &= -\text{tr}(\mathbf{B}^2) \end{aligned}$$

We can then apply some linear algebra: we'll get that for $\boxed{\rho \rightarrow 0}$ follows

$$\mathcal{I}(\rho) = \left[\text{tr}(\mathbf{A}\mathbf{R}_s\mathbf{A}^*\mathbf{R}_z^{-1}) \rho - \frac{1}{2} \text{tr}((\mathbf{A}\mathbf{R}_s\mathbf{A}^*\mathbf{R}_z^{-1})^2) \rho^2 \right] \log_2 e + o(\rho^2)$$

About this passage: it all depends on the fact that $\log \det$ is a convex function! We know the derivative of $\log \det$ and we're applying it to the fact that we're summing the eigenvalues: that's how we get the trace!

Page 74 of a linear algebra book he recommended

This is a very useful result!

If N_s, N_y are the dimensions of the vectors, for $\boxed{\rho \rightarrow +\infty}$ we get the result

$$\mathcal{I}(\rho) = \min(N_s, N_y) \log_2 \rho + \log_2 \det(\mathbf{A}\mathbf{R}_s\mathbf{A}^*\mathbf{R}_z^{-1}) + \mathcal{O}\left(\frac{1}{\rho}\right)$$

Professor said that this equation has a minor detail which is wrong, proceeded into not telling us what it was :D

Gaussian mutual information for an arbitrary constellation

Let's see the Gaussian mutual information for an arbitrary constellation.

Let s be a zero-mean unit-variance discrete random variable taking values in $s_0, \dots, s_{M-1}$ with probabilities $p_0, \dots, p_{M-1}$. This subsumes M -PSK, M -QAM, and any other discrete constellation. The PDF of $y = \sqrt{\rho}s + z$ equals

$$f_y(y) = \frac{1}{\pi} \sum_{m=0}^{M-1} p_m e^{-|y - \sqrt{\rho}s_m|^2} \quad (1.65)$$

whereas $y|s \sim \mathcal{N}_{\mathbb{C}}(\sqrt{\rho}s, 1)$. Thus,

$$\mathcal{I}^{M\text{-ary}}(\rho) = I(s; \sqrt{\rho}s + z) \quad (1.66)$$

$$= - \int f_y(y) \log_2 f_y(y) dy - \log_2(\pi e) \quad (1.67)$$

with integration over the complex plane.

$$\mathcal{I}^{M\text{-ary}}(\rho) = \log_2 M - \epsilon$$

Large ρ first order expansion

$$\log \epsilon = -\frac{d_{\min}^2}{4} \rho + o(\rho) \quad d_{\min} = \min_{k \neq \ell} |s_k - s_\ell|$$

Comments:

- In order to obtain 1.65 we used

$$f_y(Y) = \sum_{s_m \in S} f_{y|s}(Y|s_m) p_s(s_m) .$$

with $p_s(s_m) \equiv p_m$

- (1.67) is just $\mathfrak{h}(y) - \mathfrak{h}(y|s)$, considering that $y|s$ is a Gaussian RV with variance 1 and thus $\mathfrak{h}(y|s) = \mathfrak{h}(z) = \log_2(\pi e)$.
- We don't see σ_z^2 since it's $\sigma_z^2 = 1$!
- ϵ is first defined here

1.66 is the definition of mutual information. Solving the integral is pretty difficult.

What professor wants us to remember though is that

$$\mathcal{I}^{M\text{-ary}}(\rho) \xrightarrow{\rho \rightarrow \infty} \log_2(M) - \epsilon , \quad (1.68)$$

with

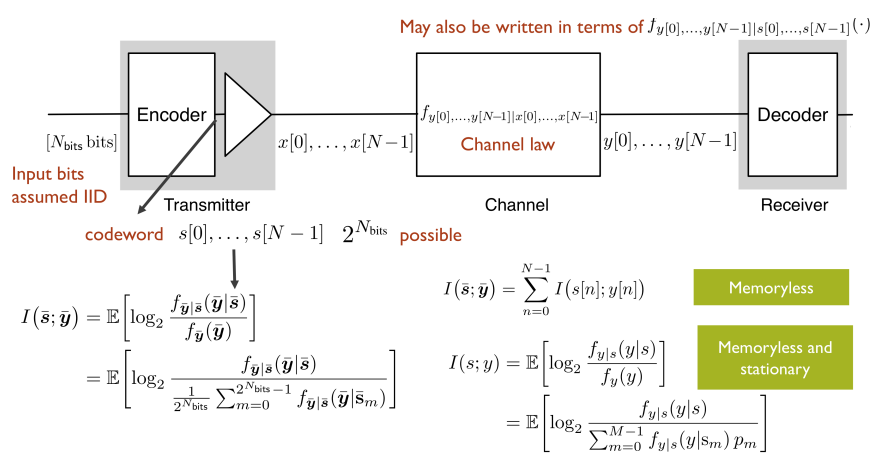
$$\log(\epsilon) = -\frac{d_{\min}^2}{4} \rho + o(\rho) : \quad (1.69)$$

intuitively, a constellation with M points asymptotically gives $\log_2(M)$ bits of information per channel use; of course this is not the case and we need to lower that term using ϵ , which brings us from the *best case scenario* to a more realistic case.

Mutual information of an encoded word

Theoretical treatment: results for generic $\bar{s}, \bar{y}$ (+for iid values and for a stationary channel)

We have an abstraction of the communication link:



We have an encoder, which maps a signal to a codebook with $2^{N_{\text{bits}}}$ words.

We send a codeword of N bits $x[n], n \in [0, N-1]$.

We then send our signal through the channel, which is characterized by noise, amplification, ...; we'll receive a whole word $y[n], n \in [0, N-1]$.

If we want to calculate the **mutual information between input words and output words**, we need to use the conditioned information function, not the joint one: we know the probability of receiving a certain message $\bar{y} = [y[0], \dots, y[N-1]]$ (as it is $f_{\bar{y}}$) when sending a certain $\bar{s} = [s[0], \dots, s[N-1]]$ and we need to work with that!

If we assume that the various symbols are equiprobable, we find that

$$f_{\bar{y}}(\bar{Y}) = \sum_{m=0}^{2^{N_{\text{bits}}}-1} f_{\bar{y}|\bar{s}}(\bar{Y}|\bar{s}_m) p_{\bar{s}}(\bar{s}_m) \stackrel{\text{equiprobable } s_m}{=} \sum_{m=0}^{2^{N_{\text{bits}}}-1} f_{\bar{y}|\bar{s}}(\bar{Y}|\bar{s}_m) \frac{1}{2^{N_{\text{bits}}}}$$

and this enables us to write

$$\begin{aligned} I(\bar{s}; \bar{y}) &= \mathcal{H}(\bar{y}) - \mathcal{H}(\bar{y}|\bar{s}) \\ &= \mathbb{E} \left[\log_2 \frac{f_{\bar{y}|\bar{s}}(\bar{y}|\bar{s})}{f_{\bar{y}}(\bar{y})} \right] \end{aligned} \quad (1.88)$$

$$= \mathbb{E} \left[\log_2 \frac{f_{\bar{y}|\bar{s}}(\bar{y}|\bar{s})}{\frac{1}{2^{N_{\text{bits}}}} \sum_{m=0}^{2^{N_{\text{bits}}}-1} f_{\bar{y}|\bar{s}}(\bar{Y}|\bar{s}_m)} \right]. \quad (1.89)$$

If the channel is **memory-less** (which might not be true: $y[1]$ might depend on $y[0]$ etc etc), the mutual information of input and output is the sum of mutual information **of every couple of input and output**: we don't have vectors anymore (and we like this!).

If this is the case, the channel law will factor out as

$$f_{\bar{y}|\bar{s}}(\cdot) = \prod_{n=0}^{N-1} f_{y[n]|s[n]}(\cdot),$$

allowing us to rewrite (1.88) as

$$I(\bar{s}; \bar{y}) = \sum_{n=0}^{N-1} I(s[n]; y[n]), \quad (1.92)$$

with

$$I(s[n]; y[n]) = \mathbb{E} \left[\log_2 \frac{f_{y[n]|s[n]}(y[n]|s[n])}{f_{y[n]}(y[n])} \right]. \quad (1.93)$$

Hidden message

Looks like one hidden message is that if the conditioned probability factorizes, we can easily rewrite the mutual information as a summation.

This follows from the fact that if the single elements are iid, then the expected value factorizes as well.

If the channel is also **stationary**, we can simplify further, as all the symbols will have the same statistics: $s \equiv s[n] \forall n$ and $y \equiv y[n] \forall n$ will hold, and can write (1.93) as

$$I(s; y) = \mathbb{E} \left[\log_2 \frac{f_{y|s}(y|s)}{f_y(y)} \right] \quad (1.94)$$

$$= \mathbb{E} \left[\log_2 \frac{f_{y|s}(y|s)}{\sum_{m=0}^{M-1} f_{y|s}(y|s_m) p_m} \right], \quad (1.95)$$

with p_m being the probability that a symbol s is s_m ; we can of course further simplify this writing if we assume equiprobability, $p_m = 1/M$.

This comes from the fact that any output does not store any information about any input other than the one which corresponds to it:

$$I(s[k]; y[n]) = \begin{cases} 0 & k \neq n \\ \text{Something} & \text{otherwise} \end{cases}.$$

Memoryless channel law with Gaussian noise

Let's see an example; if our output is defined as

$$y[n] = \sqrt{\rho} s[n] + z[n] \quad n = 0, \dots, N-1 \quad (1.96)$$

and we have that $z[0], \dots, z[N-1]$ are IID with $z \sim \mathcal{N}_{\mathbb{C}}(0, 1)$ the channel law will be the same as z , now centered in mean in the same value as the transmitted s :

$$f_{y|s}(y|s) = \frac{1}{\pi} e^{-|y - \sqrt{\rho} s|^2}. \quad (1.97)$$

The channel law is the probability that we receive y when we're transmitting s .

Capacity of a channel, information stability, spectral efficiency (+codeword error, information density)

We come to an extra important concept: **capacity**.

We can express it in different ways; we'll see **bit/s**; when we'll put the symbols one after the other.

Ideally we must study the book, in order to fully grasp these concepts

The **codeword error** probability for equiprobable words of length N is

$$p_e = \frac{1}{2^N} \sum_{m=0}^{2^N-1} P[\hat{w} \neq m | w = m],$$

where w is the transmitted word and $\hat{w}$ is the received word.

⚙️ Capacity (bits/symbol)

The capacity C in bits per symbol is the highest rate that can be achieved such that for large values of N we have

$$p_e \xrightarrow{N \rightarrow +\infty} 0.$$

The **information density** between an input word $\bar{s}$ and an output word $\bar{y}$ is defined as

$$i(\bar{s}; \bar{y}) = \log_2 \left(\frac{f_{\bar{s}, \bar{y}}(\bar{S}, \bar{Y})}{f_{\bar{s}}(\bar{S}) f_{\bar{y}}(\bar{Y})} \right).$$

If the **channel** is **information stable**, then we have that

$$\lim_{N \rightarrow +\infty} \frac{1}{N} i(\bar{s}, \bar{y}) = \lim_{N \rightarrow +\infty} \frac{1}{N} E[i(\bar{s}, \bar{y})] = \lim_{N \rightarrow +\infty} \frac{1}{N} I(\bar{s}, \bar{y}).$$

✍️ Professor's definition of information stable

A channel is information stable if the mutual information does not vary if we jump in the future or past; it's not taking the first symbols and conveying less information than the symbols coming in later.

This relation is saying that the information conveyed by $y[0], \dots, y[N-1]$ about $s[0], \dots, s[N-1]$ is invariant, provided that N is large enough

∗: note that, by **definition**,

$$\mathbb{E}[i(s; y)] = I(s; y).$$

⚙️ Sufficient condition for information stability

A sufficient condition for a channel to be information stable is that **the channel is stationary and ergodic** (ref)

✍️ Ergodicity in mean

We have a random process $x(n)$ and we know that $\mathbb{E}[x(n)] = B$.

A signal is **ergodic in mean** if

$$\lim_{N \rightarrow +\infty} \sum_{n=0}^{N-1} \frac{x(n)}{N} = B :$$

the average in time is the same as the average taken in the probability space

If the channel is stationary and ergodic, then

$$C = \max_{\text{signal constraints}} \left(\lim_{N \rightarrow +\infty} \frac{1}{N} I(\bar{s}, \bar{y}) \right).$$

The maximization is over the joint distribution of the unit-power codeword symbols $s[0], \dots, s[N-1]$.

Zero mean signals

The mean has no influence over the mutual information (and thus the capacity), but it does influence the average power spent for the transmission; for this reason, from now onwards, we'll only consider zero-mean signals.

We want the codeword error probability to go to zero for long words.

We define an **information stable channel**; we then have the information density: if it's information stable, the information density must become the mutual information for large N s.

The capacity is the **maximum mutual information which we can achieve by changing the distribution of the input signal**.

We have very long codewords and we want the probability of error on the codeword to be as low as we want.

If we have a pipe, the capacity is how much water we can push through without losing any water

In order to reach the capacity, what we can do is changing the input distribution of the symbols.

If the channel is **static, memoryless and ergodic**, then we simply have

$$C = \max_{\text{signal constraints}} I(s; y).$$

The bitrate R at which we can reliably communicate with a period T satisfies $R \leq C/T$. From the sampling theorem we have $1/T \leq B$, where B is the passband bandwidth of our transmission. By joining the two conditions, we obtain that

$$\frac{R}{B} \leq C \leftrightarrow \nu \leq C,$$

ν , spectral efficiency

where C uses the alternative measure bit/s/Hz and where the equality is reached asymptotically.

This is telling us that the capacity is not only the maximum bitrate, but also the maximum spectral capacity which we can reach.

The capacity involves searching in all the possible distributions, we'll always stay underneath it, and that's something we always need to take into account!

BICM architecture (actually: information loss when assuming bit independence, interleavers, soft decoding)

From the book:

In paragraphs before 1.5.4, the book showed that using binary encoding and constellation mapping together yields no loss of optimality. We can ask ourselves whether this also holds when we combine **soft demapping and binary decoding**.

In order to investigate this, we consider a stationary and memoryless channel as well as an M -point equiprobable constellation.

08.1_pp_3_56_A_primer_on_information_theory_and_MMSE_estimation, page 28

In this setting, codewords with IID entries are optimum and thus bits mapped to distinct symbols can be taken to be independent. However, **the channel does introduce dependencies among same-symbol bits**. Even with the coded bits produced by the binary encoder being statistically independent (as we know from past paragraphs), after demapping at the receiver dependencies do exist among the soft values for bits that traveled on the same symbol. Unaware, a binary decoder designed for IID bits ignores these dependencies and regards the channel as being memoryless, not only at a symbol level but further at a bit level [93]. **Let us see how much information can be recovered under this premise.**

First of all, we look at the decoder counterpart of (1.94), (1.95),

If the channel is also **stationary**, we can simplify further, as all the symbols will have the same statistics: $s \equiv s[n] \forall n$ and $y \equiv y[n] \forall n$ will hold, and can write (1.93) as

$$I(s; y) = \mathbb{E} \left[\log_2 \frac{f_{y|s}(y|s)}{f_y(y)} \right] \quad (1.94)$$

$$= \mathbb{E} \left[\log_2 \frac{f_{y|s}(y|s)}{\sum_{m=0}^{M-1} f_{y|s}(y|s_m) p_m} \right], \quad (1.95)$$

with p_m being the probability that a symbol s is s_m ; we can of course further simplify this writing if we assume equiprobability, $p_m = 1/M$.

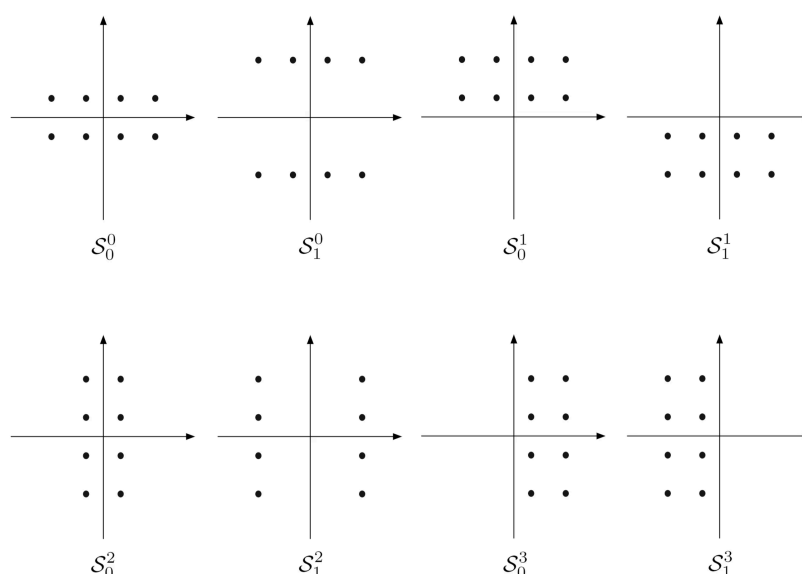
, which is

$$\sum_{\ell=0}^{\log_2(M)-1} I(b_\ell; y);$$

after much hard work, we find that it is

$$\begin{aligned} &= \sum_{\ell=0}^{\log_2 M - 1} \frac{1}{2} \left(\mathbb{E} \left[\log_2 \frac{\sum_{s_m \in S_0^\ell} f_{y|s}(y|s_m)}{\frac{1}{2} \sum_{m=0}^{M-1} f_{y|s}(y|s_m)} \right] \right. \\ &\quad \left. + \mathbb{E} \left[\log_2 \frac{\sum_{s_m \in S_1^\ell} f_{y|s}(y|s_m)}{\frac{1}{2} \sum_{m=0}^{M-1} f_{y|s}(y|s_m)} \right] \right), \end{aligned} \quad (1.137)$$

where $\mathcal{S}_0^\ell, \mathcal{S}_1^\ell$ are the subsets of constellation points whose ℓ th bit is either 0 or 1; just to give an example:



Above, 16-QAM constellation with Gray mapping. Below, subsets $\mathcal{S}_0^\ell$ and $\mathcal{S}_1^\ell$ for $\ell = 0, 1, 2, 3$ with the bits ordered from right to left.

Fig 1.4

With constellations such as BPSK and QPSK, where a single coded bit is mapped to the in-phase and quadrature dimensions of the constellation and thus **no dependencies among same-symbol coded bits**, we have that $\sum_\ell I(b_\ell; y) = I(s; y)$, since the binary decoder is not disregarding information.

Instead, **if the decoder disregards information** (acquiring information about other bits in the same symbol), the binary **coding yields** $\sum_\ell I(b_\ell; y) < I(s; y)$.

Skipped

Several examples have been skipped: 1.20, 1.21, 1.22, 1.23

Anyhow, it's true that the difference between $\sum_\ell I(b_\ell; y)$ and $I(s; y)$ is generally tiny, if the mapping of coded bits to constellation points is chosen wisely. An example of *wise choice* is Gray mapping, where neighboring constellation points only differ of one bit.

There's just one missing thing in order to have a full information-theory representation of modern transmission systems, which is **interleaving**, which generally concerns shuffling either symbols or bits in an invertible manner.

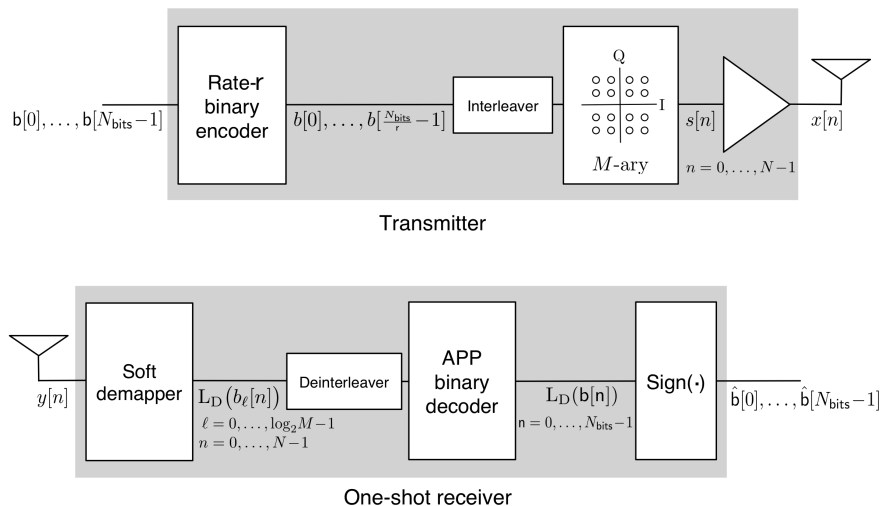
Although symbol-level interleaving would suffice to break off bursts of poor channel conditions, bit-level interleaving has the added advantage of shuffling also the bits contained in a given symbol; if the interleaving were deep enough to push any bit dependencies beyond the confines of each codeword, then the gap between $\sum_\ell I(b_\ell; y), I(s; y)$ would be closed.

It's true that interleaving has no effect when $N \rightarrow \infty$ and will thus have no effect on the mutual information calculations, but it's also true that it will improve the performance of actual codes with finite N .

The usage of Binary coding/decoding, bit-level interleaving, constellation mappers/soft demappers gives the so-called **Bit-Interleaves Coded Modulation** (BICM) architecture. BICM is information-theoretically equivalent to signal-space coding for BPSK and Gray-Mapped QPSK. For higher-order constellations, even if the dependencies among same symbol bits are not fully eradicated by interleaving and the receiver ignores them, it is only slightly inferior.

In class:

What we usually do is that we have the input bits and we do channel coding; normally, channel coding is a binary coding



Interleaver: sometimes the channel does errors in *bursts*, (it might be in a bad condition, ...); in that situation, the decoder cannot work as we'd have a long string of errors.

What we do is to *scrumble* the bits; if we have a matrix of memory, we write the values in rows

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

We read:

1, 2, 3, 4,
8, 7, 6, 5,
9, 10, 11, 12,
16, 15, 14, 13

And then

1, 5, 9, 13,
2, 6, 10, 14,
3, 7, 11, 15,
4, 8, 12, 16

This way, bits which were close are now far apart, avoiding many consecutive errors; this is a simple interleaver!

Then of course we'll do the reverse in the output.

There are many ways to implement interleavers, of course.

08.1_pp_3_56_A_primer_on_information_theory_and_MMSE_estimation, page 24

A positive L-value indicates that the bit in question is more likely to be a 1 than a 0, and vice versa for a negative value, with the magnitude indicating the confidence of the decision.

For a certain bit b , **we define the L-value** as

$$L(b) = \log \frac{P[b=1]}{P[b=0]},$$

such that if $L(b) > 0$, it's more likely that we have $b=1$ and if $L(b) < 0$ it's more likely that we have $b=0$.

The bigger the magnitude, the more certain we are about our result.

At page 08.1_pp_3_56_A_primer_on_information_theory_and_MMSE_estimation, page 25 this concept is applied to an actual message, in the form of a *soft decoder*.

This thing we've just described (together with many other much more complex things handled from page 25 to page 30) is what is called an "A Posteriori Probability" decoder or **"APP" decoder** for short.

🔗 08.1_pp_3_56_A_primer_on_information_theory_and_MMSE_estimation, page 28

Whichever the type of code, the sign of the L-values generated by an APP decoder for the message bits directly gives the MAP decisions

APP decoders are indeed really cool.

Let's say we transmit bits; we have a system which gives us values between -1 and 1; the closer we are to 1, the more we're sure that the value will be 1; the closer we are to -1, the more we're sure the value will be 0.

We then use the sign function and we have back the binary values.

🔍 I didn't get whether this is an interleaver implementation or what... 📅 2024-10-09 ⌚ 2024-10-11

From what I gathered, an **interleaver** is such to scramble bits in order to remove dependency between bits.

The rest: didn't get it at all :D

Spectral efficiency for finite-length codewords

3 Nov - 9 Nov

We can reach the Shannon bound only by pushing N to infinity; only if we're sending a *really* long codeword can we reach the capacity. We'd need to wait a long time (an *infinite* amount of time, I think) to fill that codeword and then we'd have to wait lots of latency in order to receive such a large signal.

We can show that by using small (non infinite!) N values, we don't reach the capacity and we can indeed show to which spectral efficiency we actually get!

$$\frac{R}{B} = C - \sqrt{\frac{V}{N}} Q^{-1}(p_e) + \mathcal{O}\left(\frac{\log N}{N}\right). \quad (1.143)$$

How *below* we stay depends on N, V, p_e ; we don't need to remember the capacity, but we do need to remember the fact that **it's lower when compared to the capacity**.

Takeaway points are:

1. If the **block length N increases**, we get closer to the **ideal** capacity
2. If the **probability of error decreases**, we get closer to the **ideal** capacity (since $p_e \downarrow \implies Q \uparrow \implies (1/Q) \downarrow$)
3. If the **information density has a lower variance** (totally intuitively, i.e. if input/output are more related), we get closer to the **ideal** capacity

V is the variance of the information density, defined as

$$V = \text{var}[i(s; y)]. \quad (1.144)$$